Predicting U.S. Domestic Flight Delays: A Binary Classification Approach

Robert Bagley [1]

Abstract

This paper presents a binary classification pipeline for predicting U.S. domestic flight departure delays using only information available at the time of booking. Three models — Logistic Regression, Random Forest, and XGBoost — were trained and tuned under identical conditions on a stratified sample of 500,000 flight records from the Bureau of Transportation Statistics (2019–2023). Models were evaluated using three complementary metrics: ROC-AUC, Precision-Recall AUC (PR-AUC), and Log Loss. XGBoost achieved the strongest discriminative performance (ROC-AUC: 0.7049, PR-AUC: 0.3608) while Random Forest produced better-calibrated probability estimates (Log Loss: 0.5195). All test set scores held within 0.003 of validation scores, confirming generalization without overfitting. The analysis demonstrates that meaningful delay prediction is achievable using only pre-departure scheduling features, with departure time of day emerging as the single most important predictor.

1. Introduction

Flight delays cost the U.S. economy an estimated $33 billion annually in direct and indirect costs (Federal Aviation Administration, 2020). For travelers, airlines, and airport operators alike, the ability to anticipate delays before a flight departs represents significant operational and commercial value. A reliable delay prediction model could power real-time risk scores in consumer travel applications, inform crew scheduling decisions, and enable proactive gate management at chronically delayed origin airports.

This work addresses the binary classification task of predicting whether a scheduled U.S. domestic flight will depart with a delay of 15 or more minutes — the threshold used by the FAA and the Department of Transportation (DOT) for reporting purposes. The key constraint is that the model may only use information available at the time of booking or on the day of departure, prior to taxi. Post-flight information — including actual departure times, arrival delay, and delay cause attributions — is explicitly excluded to prevent data leakage.

Three models of increasing complexity are trained and compared: Logistic Regression as an interpretable linear baseline, Random Forest as a non-linear addition, and XGBoost as an iterative gradient boosting method. All three are tuned under identical conditions using `RandomizedSearchCV` and evaluated on a held-out test set that is touched exactly once.

This project is the machine learning companion to a parallel data visualization study (Bagley, 2026) that analyzes the same dataset through exploratory visualization, surfacing temporal, geographic, and operational delay patterns that inform the feature engineering decisions made here.

2. Dataset

2.1. Source and Scale

The dataset is the Flight Delay and Cancellation Dataset 2019–2023, sourced from the U.S. Department of Transportation's Bureau of Transportation Statistics and distributed via Kaggle (Zel, 2023). The raw file contains approximately 3 million rows and 32 columns covering all U.S. domestic commercial flights across five full calendar years.

2.2. Sampling

The full dataset is computationally prohibitive for cross-validated hyperparameter search across three model families. A stratified sample of 500,000 flights was drawn using `scikit-learn`'s `train_test_split` with `stratify=IS_DELAYED`, preserving the natural 82/18 class imbalance ratio 1. Statistical power analysis confirms that 500,000 stratified samples provide sufficient power to detect meaningful performance differences

Course project for DASC 5431 — Data Analytics & Machine Learning, University of Houston–Clear Lake, Spring 2026.
[1]College of Science and Engineering, University of Houston–Clear Lake, Houston, Texas, USA. Correspondence to: Robert Bagley <bagleyr6515@uhcl.edu>.

between models at conventional significance levels.

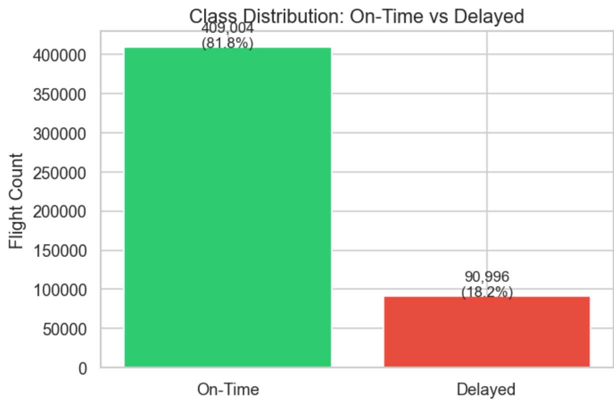

Figure 1. Class imbalance

2.3. Data Preparation

Leakage removal. The single most critical preparation step is the removal of all post-flight columns before any other processing. Twenty-two columns were dropped, including actual departure and arrival times, taxi times, wheels-off and wheels-on timestamps, actual elapsed time, air time, and all five delay cause attribution columns (`DELAY_DUE_CARRIER`, `DELAY_DUE_WEATHER`, `DELAY_DUE_NAS`, `DELAY_DUE_SECURITY`, `DELAY_DUE_LATE_AIRCRAFT`). These columns are only observable after a flight lands and constitute direct leakage of the outcome being predicted.

Target construction. The binary target `IS_DELAYED` is constructed from `DEP_DELAY` using the FAA/DOT standard:

$$\texttt{IS\_DELAYED}_i = [\texttt{DEP\_DELAY}_i \geq 15] \qquad (1)$$

The source column `DEP_DELAY` is immediately dropped after target construction to prevent target leakage. Canceled flights (null `DEP_DELAY`) are excluded from the dataset as they represent a different outcome.

Class distribution. The resulting dataset exhibits a 4.49:1 class imbalance ratio: approximately 82% on-time (class 0) and 18% delayed (class 1).

Table 1 summarizes the dataset attributes.

3. Feature Engineering

All features used in modeling are derivable from information available before departure. Table 2 lists the final feature set.

Temporal feature extraction. `HOUR_OF_DAY` is extracted from the scheduled departure time (`CRS_DEP_TIME`) and serves as the strongest individual predictor, consistent with the domino effect documented in the companion visualization study (Bagley, 2026) in the shared graph Figure 2. `MONTH`, `DAY_OF_WEEK`, and `YEAR` are extracted from `FL_DATE`.

Table 1. Dataset Summary

Attribute	Detail
Source	BTS via Kaggle
Time Span	January 2019 – December 2023
Raw Size	∼3 million rows, 32 columns
Working Sample	500,000 rows (stratified)
Class Ratio	82% On-Time / 18% Delayed
Imbalance Ratio	4.49:1
Columns Dropped	22 (leakage + redundant)

Table 2. Final Feature Set

Feature	Type	Source
HOUR_OF_DAY	Numeric	CRS_DEP_TIME
CRS_ELAPSED_TIME	Numeric	Raw column
MONTH	Numeric	FL_DATE
DAY_OF_WEEK	Numeric	FL_DATE
YEAR	Numeric	FL_DATE
AIRLINE_CODE	Categorical	Raw column
ORIGIN	Categorical	Raw column
DEST	Categorical	Raw column
SEASON	Categorical	MONTH
TIME_BUCKET	Categorical	HOUR_OF_DAY

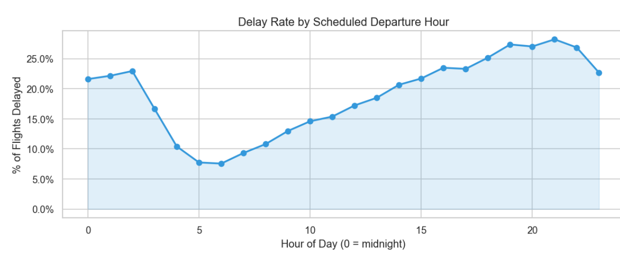

Figure 2. Departure delay rate by scheduled departure hour with ±1 standard deviation confidence band. The domino effect is visible as a near-linear climb from 7AM onward.

Categorical groupings. `SEASON` maps months to four seasonal categories. `TIME_BUCKET` maps departure hours to five named periods (Red-Eye, Morning, Afternoon, Evening, Night), providing a coarser but explicitly categorical representation of the time signal alongside the continuous `HOUR_OF_DAY`.

Multicollinearity removal. Correlation analysis revealed three near-perfectly collinear feature pairs: `CRS_DEP_TIME` with `HOUR_OF_DAY` (r = 1.00), `DISTANCE` with `CRS_ELAPSED_TIME` (r = 0.98), and `CRS_ARR_TIME` with `CRS_DEP_TIME` (r = 0.70). In each pair, the more interpretable feature was retained and the redundant column dropped. *See Figure 3*

Redundancy removal. `IS_WEEKEND` was dropped despite being engineered ($r = 0.78$ with `DAY_OF_WEEK`) due to near-zero correlation with the target ($r = 0.01$). Both `AIRLINE` and `AIRLINE_CODE` were initially retained but post-modeling feature importance analysis revealed that both appeared in the top features for the same carrier, confirming redundancy. `AIRLINE` was subsequently dropped in favor of `AIRLINE_CODE`.

4. Methods

4.1. Data Partitioning

The 500,000-row sample was split into training (70%), validation (15%), and test (15%) partitions using stratified splitting:

$$N_{\text{train}} = 350{,}000 \quad N_{\text{val}} = 75{,}000 \quad N_{\text{test}} = 75{,}000 \quad (2)$$

Stratification on `IS_DELAYED` preserves the 82/18 class ratio across all three partitions. The test set was held out and evaluated exactly once, after final model selection on the validation set.

4.2. Preprocessing Pipeline

All preprocessing was implemented using `scikit-learn`'s `Pipeline` and `ColumnTransformer` to ensure that all transformers are fit exclusively on training data and applied to validation and test partitions without refitting — preventing any form of preprocessing leakage.

Numeric features pass through a two-step transformer: median imputation followed by `StandardScaler` (zero mean, unit variance). Scaling is required for Logistic Regression, which uses gradient descent and is sensitive to feature magnitude differences.

Categorical features pass through mode imputation followed by `OneHotEncoder` with `handle_unknown='ignore'`, which maps unseen categories in the validation or test set to an all-zero vector rather than raising an error.

Note: No actual imputation was conducted as the sample data was not missing any values.

4.3. Models

Three models of increasing complexity were trained:

Logistic Regression serves as the interpretable linear baseline. Class imbalance is addressed via `class_weight='balanced'`, which inversely weights classes proportional to their frequency.

Random Forest is a parallel group of decision trees, each trained on a bootstrap sample with a random feature subset. Class imbalance is again addressed via `class_weight='balanced'`.

XGBoost is a sequential gradient boosting method in which each successive tree corrects the residual errors of the ensemble. Class imbalance is addressed via `scale_pos_weight`, set to the negative-to-positive class ratio:

$$\texttt{scale\_pos\_weight} = \frac{N_{\text{on-time}}}{N_{\text{delayed}}} = \frac{410{,}502}{91{,}498} \approx 4.49 \quad (3)$$

GPU acceleration was enabled for XGBoost training via `device='cuda'` and `tree_method='hist'` on an NVIDIA RTX 3060 (16GB VRAM).

4.4. Hyperparameter Search

All three models were tuned using `RandomizedSearchCV` under identical conditions: `n_iter=5`, `cv=3`, `scoring='roc_auc'`, `random_state=42`. Table 3 lists the search spaces. Models were tuned on the training partition only. The validation set was used exclusively for post-tuning evaluation and model selection.

Table 3. Hyperparameter Search Spaces

Model	Parameter	Values Searched
LR	C	[0.01, 0.1, 1, 10]
LR	solver	[lbfgs, saga]
LR	penalty	[l2]
RF	n_estimators	[100, 200, 300]
RF	max_depth	[None, 10, 20]
RF	min_samples_split	[5, 5, 25]
XGB	n_estimators	[200, 300, 400]
XGB	learning_rate	[0.01, 0.05, 0.1]
XGB	max_depth	[4, 6, 8]
XGB	subsample	[0.7, 0.8, 1.0]
XGB	colsample_bytree	[0.7, 0.8, 1.0]

4.5. Evaluation Metrics

Three complementary metrics were used for evaluation:

ROC-AUC measures the model's ability to rank delayed flights above on-time flights across all classification thresholds. A random classifier scores 0.50. ROC-AUC is less sensitive to class imbalance because False Positive Rate is computed against the majority class.

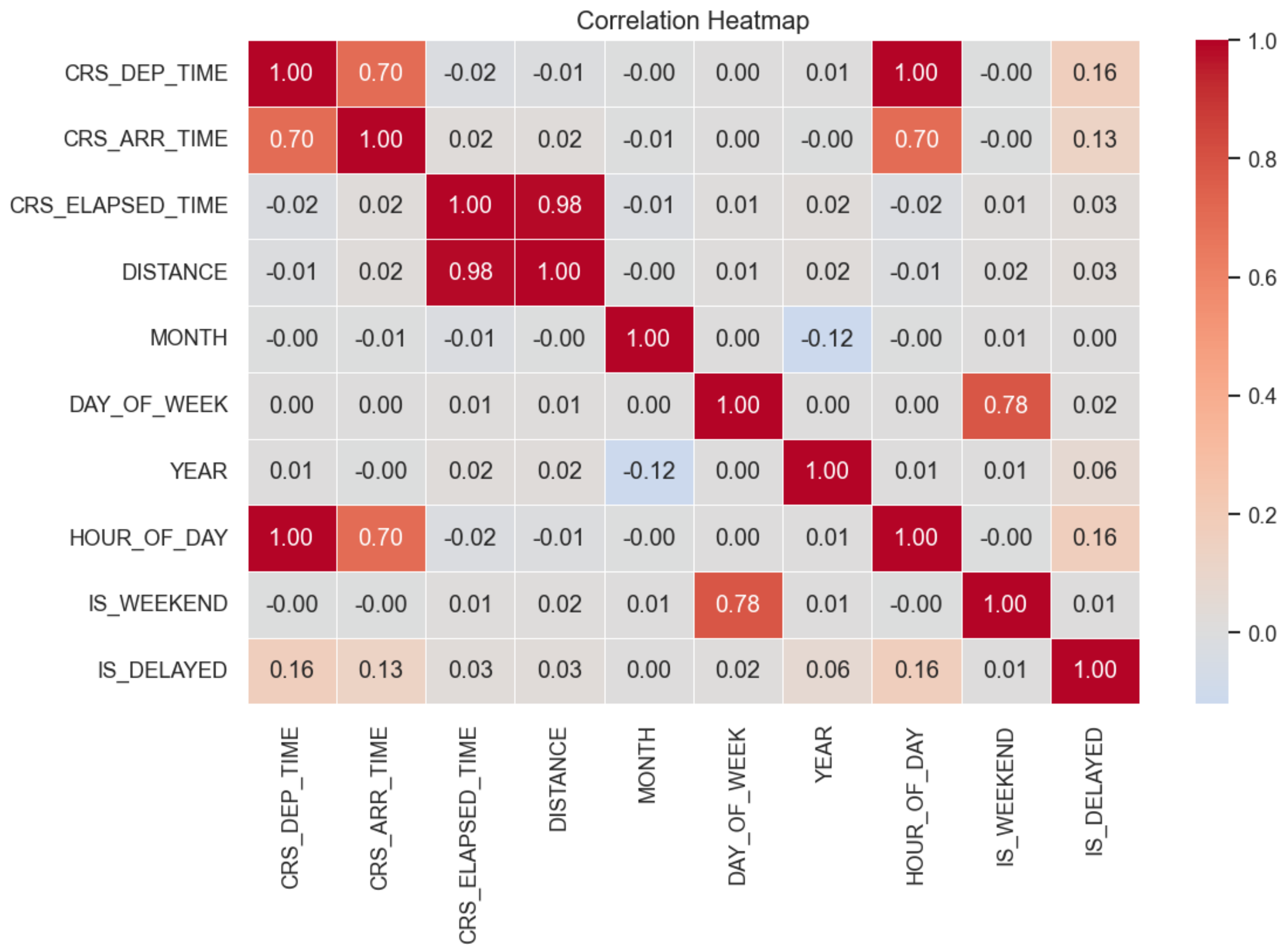

Figure 3. Correlation Heatmap of preliminary features with post-flight features removed.

PR-AUC (Average Precision) measures precision-recall trade-off specifically on the minority (delayed) class. A random classifier on this dataset scores approximately 0.18 — the base rate of positive examples. PR-AUC is more sensitive to class imbalance and provides a more honest assessment of minority class performance.

Log Loss measures the quality of probability calibration, penalizing confident incorrect predictions more severely than uncertain ones. Lower is better; a perfectly calibrated model minimizes Log Loss.

Together these three metrics cover ranking ability, minority class precision, and probability calibration — three distinct and complementary dimensions of classifier quality.

5. Results

5.1. Model Comparison — Validation Set

Table 4 presents the validation set performance of the best-tuned version of each model across all three metrics.

Table 4. Validation Set Results (tuned models)

Model	ROC-AUC	PR-AUC	Log Loss
XGBoost	**0.7073**	**0.3636**	0.6209
Random Forest	0.6944	0.3530	**0.5195**
Logistic Regression	0.6725	0.3182	0.6473

XGBoost achieves the highest ROC-AUC and PR-AUC (Figure 4), making it the clear selection for final evaluation. However, Random Forest produces the lowest Log Loss, indicating better-calibrated probability estimates. This distinction is meaningful in production contexts where the raw probability score — rather than just the binary prediction — is surfaced to end users.

5.2. Final Test Set Results — XGBoost

Table 5 presents the final evaluation of the selected XGBoost model on the held-out test set alongside the corresponding validation scores.

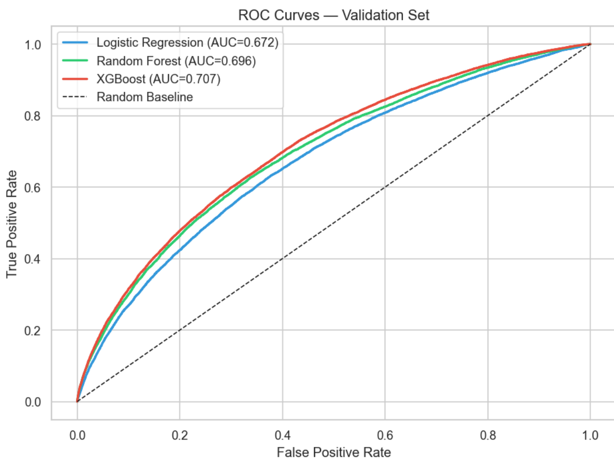

Figure 4. ROC Curves - Validation set

Table 5. Final Test Set Results — XGBoost

Metric	**Validation**	**Test**	**Drift**
ROC-AUC	0.7073	0.7049	−0.0024
PR-AUC	0.3636	0.3608	−0.0028
Log Loss	0.6209	0.6225	+0.0016

All three metrics hold within 0.003 of their validation scores, confirming that the model generalizes without overfitting. The PR-AUC of 0.3608 represents approximately double the random classifier baseline of ∼0.18, indicating meaningful discriminative power on the minority class despite the 4.49:1 imbalance.

5.3. Classification Report

At the default 0.5 classification threshold, the final XGBoost model achieves the following per-class performance on the test set:

Table 6. Classification Report — Test Set

Class	**Precision**	**Recall**	**F1**	**Support**
On-Time	0.89	0.65	0.75	61,351
Delayed	0.29	0.64	0.40	13,649
Weighted Avg	0.78	0.65	0.69	75,000

The model is calibrated toward high recall on the delayed class (0.64) at the cost of precision (0.29). This trade-off is appropriate for a traveler-facing application where missing a delay (false negative) carries a higher cost than a false alarm (false positive). On-time precision of 0.89 indicates that when the model predicts a flight will be on time, it is correct 89% of the time.

5.4. Feature Importance

Figure 5 presents the top 20 feature importances from the final XGBoost model, extracted after one-hot encoding of categorical features.

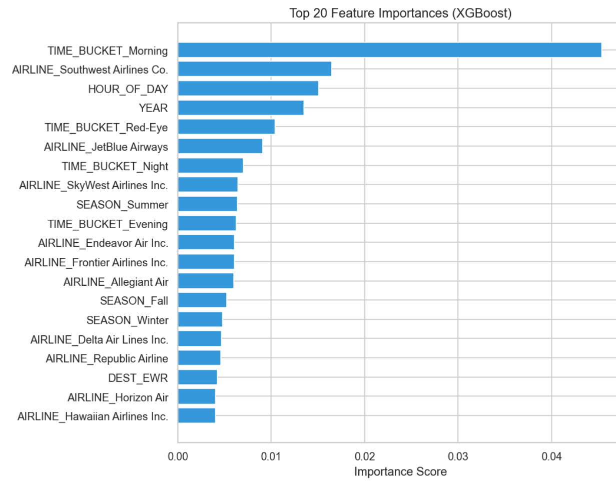

Figure 5. Top 20 feature importances from the final XGBoost model.

`TIME_BUCKET_Morning` is the single most important feature at approximately 4× the importance of the second-ranked feature, consistent with the domino effect pattern documented in the companion visualization analysis — morning departures are the reset point of the daily schedule, and their on-time performance propagates through the entire day. `HOUR_OF_DAY` ranks second, reflecting the continuous time-of-day signal alongside the categorical bucket representation.

Airline identity features appear prominently: Southwest Airlines (WN) appears in the top five, while JetBlue (B6), Allegiant (G4), and Frontier (F9), confirmed worst performers, and Endeavor Air (9E), a stellar performer, all appear in the top 20 — consistent with the companion visualization study (Bagley, 2026) (*See Figure* 6 below). This cross-validation between the two projects strengthens the reliability of both findings.

`SEASON_Summer` appears in the top 20 while winter and spring seasons do not, suggesting that summer's convective weather patterns produce a sufficiently distinct delay signal that the model learns it as an explicit feature, while other seasons are captured implicitly through temporal features.

6. Discussion

6.1. Correct Tuning Workflow

A methodological priority in this work was ensuring a fair comparison across all three models. An earlier version of

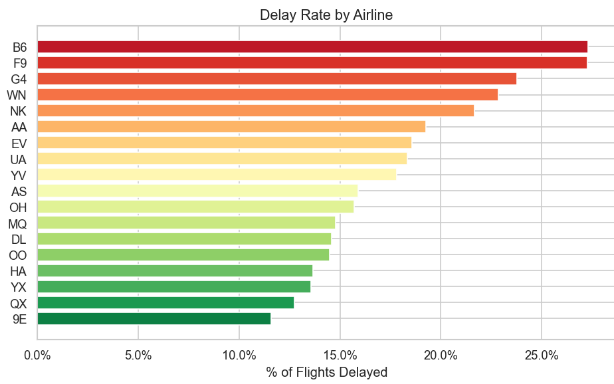

Figure 6. Ranking airlines by delay rate — worst to best

this analysis tuned only XGBoost after model comparison, leaving Logistic Regression and Random Forest in their default configurations. This is a common but significant error — comparing a tuned model against untuned baselines conflates the effect of the model family with the effect of tuning, making it impossible to attribute performance differences to the model architecture rather than the optimization effort applied to it.

The corrected workflow tunes all three models under identical conditions — same training data (350,000 rows), same cross-validation scheme (`cv=3`, `n_iter=5`), same random state — before comparing their best versions. This ensures that any observed performance difference reflects the model's inherent capacity rather than a tuning advantage.

6.2. Compute Environment

An important practical finding of this project is the impact of the compute environment on feasibility. Initial development in Google Colab (free tier, 2 vCPUs) produced unacceptably long tuning runtimes — Logistic Regression alone ran for over 30 minutes before the session was interrupted. Switching to a local machine with 16 logical processors and an NVIDIA RTX 3060 GPU reduced total tuning time to approximately 22 minutes across all three models, with XGBoost completing in under 2 minutes via GPU acceleration.

This finding has practical implications for reproducibility: code that is computationally feasible on a capable local machine may be impractical on free cloud compute resources, and project timelines should account for this discrepancy.

6.3. Pandas Version Compatibility

A non-trivial debugging challenge arose from differences between the pandas version available in Google Colab and the version installed in the local virtual environment. In newer versions of pandas, `groupby().apply()` excludes the groupby key column from the result by default, silently dropping the `IS_DELAYED` column after stratified sampling. This produced a `KeyError` that was not present in Colab, where an older pandas version preserves the key column. The fix — replacing the `groupby().apply()` pattern with a `train_test_split` trick — is more robust across versions and is now the recommended approach. Additionally, `.copy()` must be appended to any pandas operation that returns a view (`.dropna()`, `.drop()`, boolean filters) to prevent silent `SettingWithCopyWarning` failures.

7. Limitations and Future Work

7.1. Limitations

Absence of weather variables. Real-time weather conditions — wind speed, precipitation, visibility, convective activity — are among the strongest known predictors of flight delays but are not available in the BTS dataset. The `DELAY_DUE_WEATHER` column records after-flight delay attributions rather than actual meteorological observations and was excluded as leakage. Integrating historical weather data by airport and date would likely produce a meaningful improvement in predictive performance.

COVID-19 structural break. The 2019–2023 training window includes the 2020 COVID-19 period, during which flight volumes fell by over 60% and delay patterns were fundamentally atypical. Training on pre-COVID and post-COVID data simultaneously may reduce the model's accuracy for either period individually and could introduce non-stationarity into the learned representations. While this was an interesting topic for investigation in the data analysis report, the atypical pattern here may have negative effects on the classification models.

7.2. Future Work

Weather data integration. Joining NOAA historical weather observations by airport and date would introduce the strongest missing predictor class and likely produce a substantial improvement in PR-AUC.

LightGBM comparison. LightGBM is a competitive alternative to XGBoost on tabular classification tasks that was not included in this study. Adding it as a fourth model would complete the gradient boosting comparison and potentially improve final performance.

8. Conclusion

This work demonstrates that meaningful binary classification of U.S. domestic flight departure delays is achievable using only pre-departure scheduling features — without access to real-time weather, air traffic, or operational data. XGBoost achieves a test ROC-AUC of 0.7049 and PR-AUC of 0.3608, representing approximately double the random classifier baseline on the more demanding minority-class metric. All test scores hold within 0.003 of validation scores, confirming generalization without overfitting.

The most important methodological contribution of this work is the corrected hyperparameter tuning workflow: all three models were tuned under identical conditions before comparison, ensuring that observed performance differences reflect model architecture rather than differential tuning effort. The feature importance analysis confirms the primacy of temporal features — particularly departure time of day — consistent with the domino effect documented in the companion visualization study, providing cross-project validation of both analyses.

Flight delay prediction with this feature set has direct application in consumer travel recommendation, airline scheduling optimization, and airport capacity planning. The gap between the current PR-AUC of 0.36 and a production-quality system can likely be substantially closed by integrating real-time weather variables — an obvious next step for future work.

References

Bagley, R. Cleared for takeoff? a visual analysis of u.s. domestic flight delay patterns (2019–2023). Technical report, University of Houston–Clear Lake, 2026. DASC 5231 Data Analysis & Data Visualization Final Project.

Federal Aviation Administration. Cost of delay estimates. Technical report, U.S. Federal Aviation Administration, 2020. URL https://www.faa.gov/data_research/aviation_data_statistics/media/cost_delay_estimates.pdf.

Zel, P. Flight delay and cancellation dataset 2019–2023, 2023. URL https://www.kaggle.com/datasets/patrickzel/flight-delay-and-cancellation-dataset-2019-2023. Data sourced from U.S. DOT Bureau of Transportation Statistics.